



SAPIENZA
UNIVERSITÀ DI ROMA

DIPARTIMENTO DI INGEGNERIA INFORMATICA E AUTOMATICA

Lumen Drift

INTERACTIVE GRAPHICS

Professors:

Marco Schaerf

Students:

Fabio Falconi

2048079

Academic Year 2025/2026

Contents

1	Introduction	3
1.1	Libraries Used	3
1.2	Other Tools Used	3
1.3	Browser Compatibility	4
2	User Manual	5
2.1	Premise of the game	5
2.2	Exploration Phase	5
2.2.1	Introduction and controls	5
2.2.2	Interacting with objects	6
2.2.3	Light Puzzle	6
2.3	Platforming Phase	7
2.4	The Victory Scene	7
3	Models	8
3.1	The Player: Hierarchical model	8
3.2	Celestial Bodies	9
3.3	Interactive Elements	11
3.4	Textures	12
4	Environment	13
4.1	The Command Vessel	13
4.2	The Black Hole	13
4.3	Galactic Clusters	14
4.4	Sci-Fi Furnishings	14
4.5	Structural Foundation: Stars, Platforms, and Walls	14
4.6	Textures	15
5	Animations	16
5.1	Player Movements	16
5.2	The Solar System Simulation	16
5.2.1	The Binary Planet System	17
5.2.2	Elliptical Comet Orbit and Tail Animation	17
5.3	Black Hole Disk Dynamics	19
5.4	Gravity and collisions	20
5.5	Camera Management and Anti-Clipping System	20
5.6	Environmental and UI Animations	21

6	Lighting	23
6.1	Global Illumination and Atmospheric Lighting	23
6.2	Local and Decorative Lighting	23
6.3	Dynamic Player-Controlled Lighting (The Flashlight)	24
6.4	Physical Lighting Logic and Intensity Computation	24
7	Audio	26
7.1	Ambient Music	26
7.2	Positional Audio	26
7.3	Sound Effects	26

1 Introduction

This document describes the architecture, design choices, and user guidelines for an interactive, browser-executable three-dimensional graphics application developed as the final project for the *Interactive Graphics* course.

The application is a virtual environment for space and planetary exploration in which the user controls an avatar in third person view. The game world features a spaceship inside a simulated star system, which orchestrates the orbital movement of six celestial bodies and their respective satellites. Beyond the astronomical simulation, the environment includes real-time interaction mechanics: a physics system managing gravity and jumping, the interaction of the player's torch with environment elements, interaction with the spaceship elements.

1.1 Libraries Used

The following open-source JavaScript libraries were used to develop the application:

- **Three.js:** The primary graphics engine of the application. It was used as an object-oriented wrapper over WebGL to manage the scene graph (`THREE.Scene`).
- **GLTFLoader:** A module from the Three.js ecosystem used for the loading of 3D assets in the standard *glTF* format (`.gltf` or `.glb` extensions). It allowed efficient loading of planetary models and game structures while preserving texture coordinates and native materials.
- **Tween.js (@tweenjs/tween.js):** A tweening library used to handle interpolation of transformation matrices. It controls mainly the progressive movement of the main room's door.

1.2 Other Tools Used

The following auxiliary tools were used to support the development pipeline, asset preparation, and project delivery:

- **Visual Studio Code:** The IDE used for writing and formatting the JavaScript source code, with local debugging via browser Developer Tools.
- **Blender 3.x:** An open-source 3D modeling tool used to produce and scale the planetary meshes before exporting them to `.gltf` format, ensuring no embedded animations were included.
- **GitHub Pages:** A static hosting service used to publicly deploy the web application, making it directly accessible to the evaluator via a URL provided in the `README.md` file.

- **Overleaf:** Online LaTeX editor used to write this document.

1.3 Browser Compatibility

The application relies on modern web standards, so it requires a desktop browser with hardware support for **WebGL 2.0** and shadow map extensions.

The application has been tested, verifying the handling of window resize events and keyboard input (WASD + Shift), on the following browser versions:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

2 User Manual

This section describes how the user can interact with our application. The core of the game focuses on an exploration and platforming phase, where the user must navigate the environment, solve environmental puzzles utilizing light mechanics, and finally retrieve a power source to complete the game.

2.1 Premise of the game

The user plays the role of an android unit awakened during an emergency in a spaceship. The spaceship is suffering severe structural failures due to its proximity to a black hole's event horizon. During the incident, the ship's reactor was ejected into the external sector of the hull. The goal of the game is to read the system logs to understand the situation, find a way to open the sealed airlock by solving a lighting puzzle, navigate a parkour section outside the ship, and recover the ejected reactor to place it back onto the central core pedestal.

2.2 Exploration Phase

2.2.1 Introduction and controls

The game is played from a third-person perspective with a dynamic camera that automatically avoids clipping into walls. The user can move the character and look around the environment using the following control scheme:

- **WASD keys:** Move forward, left, backward, and right relative to the character's orientation.
- **Mouse:** Drag the mouse to rotate the camera and dictate the character's facing direction.
- **Spacebar:** Jump over obstacles and gaps.
- **Shift:** Hold this key while moving to sprint at a higher speed.
- **E:** Toggle the android's built-in flashlight on and off.
- **F:** Interact with objects, terminals, and key items in the environment.
- **Esc:** Pause the game, opening a menu that freezes player input and provides a slider to adjust the audio volume. Pressing *Resume* returns to the game.

2.2.2 Interacting with objects

Several interactive elements are present in the spaceship. The user will be notified by an on-screen prompt reading *"Press [F] to interact"* whenever they are close enough to a valid object. The interactive objects include:

- **Sci-Fi Consoles:** Approaching the main terminal and pressing F opens the system UI, which displays emergency logs and crucial hints regarding the environment and the puzzles. Pressing F again closes the terminal.
- **Buttons:** Wall-mounted switches that can be pressed to toggle environmental elements, such as emergency warning lights.
- **The Reactor:** The primary objective of the mission. Pressing F while near the reactor allows the player to pick it up.
- **The Core Pedestal:** Once the reactor is collected, interacting with the central pedestal in the starting room will place the reactor back into its housing, restoring power to the ship.

2.2.3 Light Puzzle

A unique feature of this application is the interaction between light sources and the environment. Certain materials debris react in a special way to light.

Sensors and the Main Door: To exit the initial room, the player must bypass a sealed door. As hinted by the console logs, this requires overloading two photo-sensitive crystals at the exact same time. Since the player only has one flashlight, they must use a wall button to activate a fixed environmental light pointing at the first crystal, and then manually aim their own flashlight (using the **E** key) at the second crystal. Once both are illuminated simultaneously, the door opens.

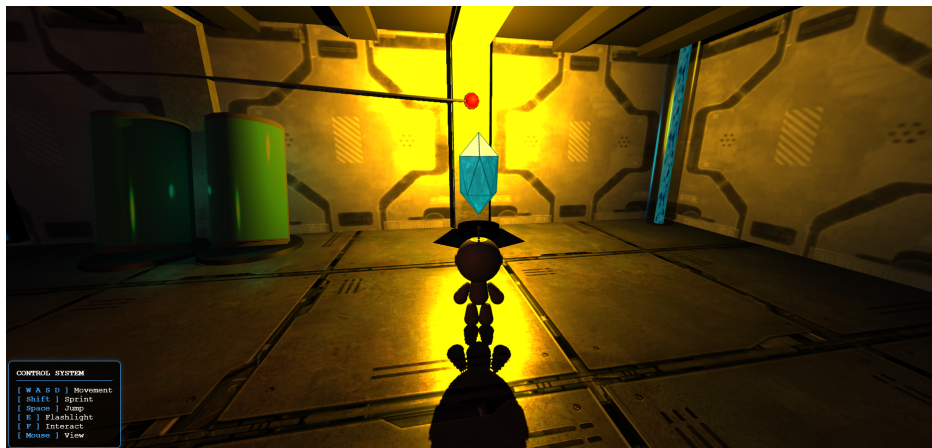


Figure 1: Activated Light Sensor

2.3 Platforming Phase

In the external sector, the player must go through a parkour challenge jumping across floating debris. The flashlight is essential here, as the platforms react dynamically to illumination:

- **Light-only platforms:** These platforms are invisible and lack collision by default. They materialize and become solid only when hit by a light source.
- **Shadow platforms:** These platforms are solid in the dark but will vanish if illuminated by the flashlight.

The player must carefully toggle their flashlight on and off to successfully navigate this section. In the end he will find the lost reactor, and will be able to retrieve it and bring it back to the initial room.

2.4 The Victory Scene

Once the player successfully retrieves the reactor from the external debris field and places it back onto the central pedestal, the game transitions into the ending scene. The user loses control of the character as a cinematic sequence starts. The camera detaches from the player and begins to smoothly orbit around the protagonist. At the same time, the android performs a victory animation by happily swinging its arms. Finally, a futuristic, holographic *"Mission Accomplished"* UI fades into the center of the screen, notifying the player that the game is completed.



Figure 2: Victory animation

3 Models

Models are divided into three core categories: meshes found online and then customized, original assets generated via Blender and runtime-generated geometries orchestrated through Three.js. All external assets have no animation data, focusing exclusively on geometry.

3.1 The Player: Hierarchical model

The player character represents the primary complex hierarchical model within the application. The original monolithic asset was sourced from Sketchfab and subsequently imported into Blender to undergo a complete reorganization.

To enable software motion without importing pre-recorded animations, the model's unified mesh was severed into distinct functional components. The geometry was separated into five core meshes: the Torso (acting as the main body and root node), the LeftArm, the RightArm, the LeftLeg, and the RightLeg.

Then, a local kinematic chain was established using a parent-child relationship via Blender's *Object (Keep Transform)* function. This hierarchy guarantees that any global translation or rotation applied to the Torso is natively propagated to the limbs through spatial matrix concatenation, while simultaneously allowing each limb to execute independent movements relative to its local joint origin during the walk cycle.

Figure 3 illustrates the topology of the character mesh, mapped directly into the application's Scene Graph via the GLTFLoader:

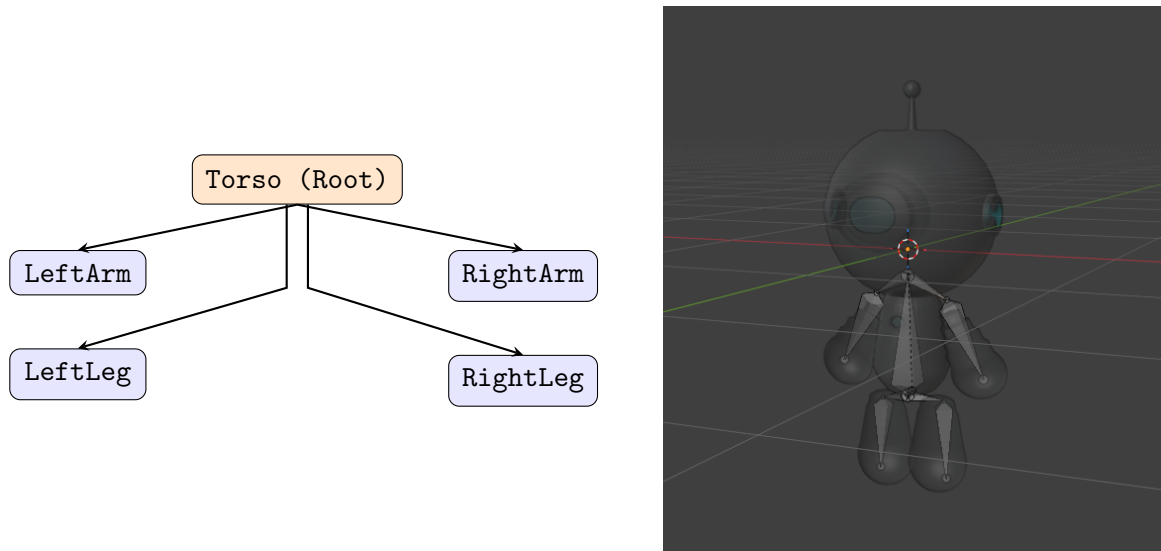


Figure 3: Hierarchical structure of the player character. The tree diagram (left) maps the parent-child dependencies from the central torso root node, matching the structure displayed in the rigged viewport (right).

3.2 Celestial Bodies

The background environment features a diverse set of planets in a solar system. All major planetary assets were modeled from scratch within Blender using sculpting and material shading techniques.

- **Standard Planets and Satellites:** Two classic planets and one natural satellite (moon) were modeled using geometric UV spheres subjected to Blender’s *Sculpt Mode*. Brushes were utilized to deform the vertices, generating surface irregularities and craters.

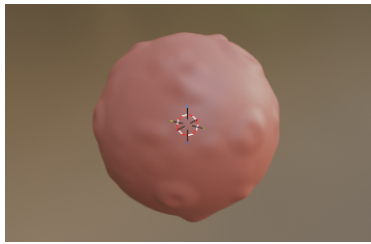


Figure 4: Basic Planet

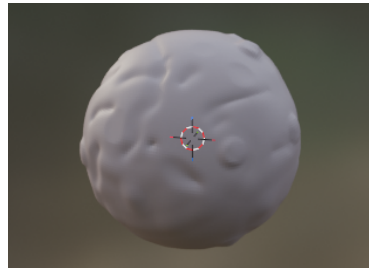


Figure 5: Moon

- **Shattered Planet:** A destroyed, fragmented planet was engineered utilizing Blender’s *Cell Fracture* add-on. A base sphere was subjected to a localized Voronoi-based algorithm, fracturing the mesh into multiple shards. These shards are grouped together to maintain the shape while allowing individual movements. In the application, a purple pointlight was put at the center of the planet to highlight the fractures.

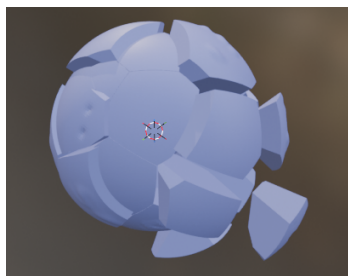


Figure 6: Shattered Planet

- **Ringed Planet:** A Saturn-like celestial body was constructed by pairing a central sculpted sphere with a concentric, flattened Torus geometry representing the planetary ring. The ring features a custom gradient texture.

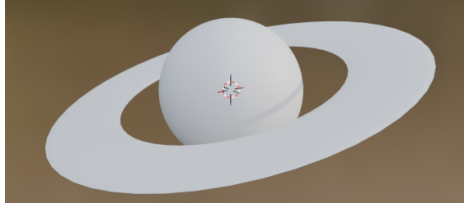


Figure 7: Planet with Rings

- **The Volcanic Lava Planet:** This binary companion planet was created using a layered structure. The inner core consists of a bright orange emissive sphere acting as magma. Over this core, a slightly larger secondary sphere, representing the solid crust, was wrapped. Random triangular faces were manually deleted from this superstructure to simulate floating continental plates drifting on top of the liquid lava. Additionally, procedural cones were extruded on the surface to act as active volcanos.
- **The Glacial Planet:** The second component of the binary system simulates extreme cold. A pale-blue emissive sphere was used as the base. The *Cell Fracture* tool was again employed to create deep cracks across the surface. Independent crystal meshes were randomly dispersed and scattered across the surface to act as ice formations.

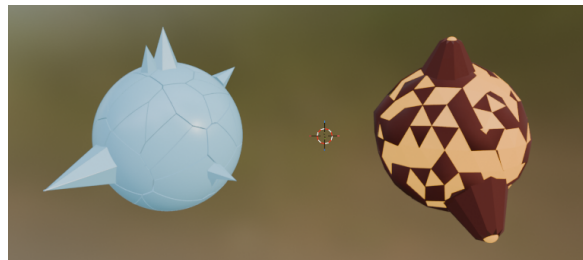


Figure 8: Binary planet system - Volcanic and Glacial planets

- **The Comet:** The comet's core was modeled by applying irregular, highly asymmetrical sculpting patterns onto a small sphere to mimic a coarse asteroid nucleus. A group of particles is attached to the back of the comet to simulate its tail.

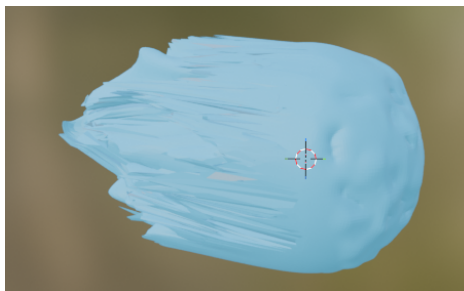


Figure 9: Comet

- **The Satellite:** An artificial satellite orbiting one of the standard planets, made in Blender with standard geometry. Unlike the natural moon, it is a man-made structure featuring a central body, lateral solar panels and a top antenna. A small spherical element at the top of the antenna is used in the code as a blinking beacon light to make the small satellite more visible.

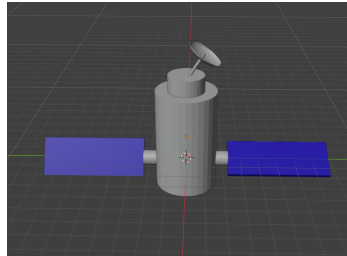


Figure 10: Artificial Satellite

3.3 Interactive Elements

The remaining assets within the environment serve specific gameplay interactions or define the map boundaries:

- **Light Sensors:** The sensor models were crafted directly in Blender using sci-fi geometry. The crystalline upper component was modeled as an independent object detached from the mechanical pedestal base. This structural separation allows the script to apply continuous spinning transformations to the crystal on its axis whenever its state is updated.
- **Interactive Buttons:** A sci-fi button prop was sourced from Sketchfab and refactored. The mesh was segmented to isolate the movable Key from the static Base. This optimization allows the TWEEN.js engine to interpolate the key's Z-depth independently, animating the button pressing giving visual feedback whenever a player interacts. The button triggers an alarm light, and the alarm's model also was taken from SketchFab.
- **The Reactor:** A sci-fi power source that the player has to retrieve. It can be interacted to be picked up, outside the spaceship. The model was crafted directly in Blender.

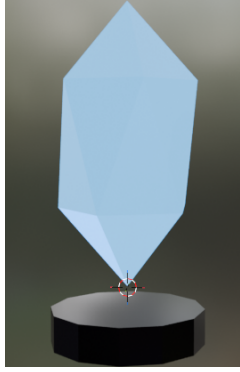


Figure 11: Light Sensor Model

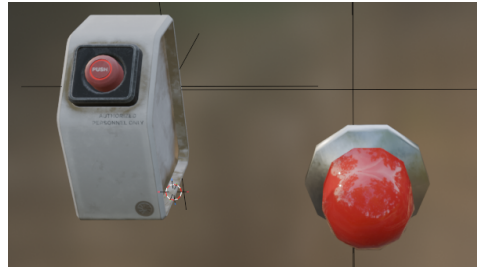


Figure 12: SketchFab Models

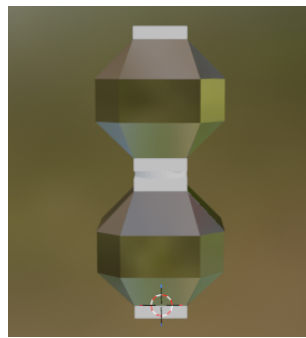


Figure 13: Reactor Model

3.4 Textures

The textures applied on the models are automatically included in the model itself, since the GLTFLoader library is used and GLT files have an included texture. The SketchFab models use, therefore, the texture that was downloaded with them. The Blender models, instead, use the texture that was realized in for them in Blender.

4 Environment

The environment is architected to produce visually pleasing objects while also achieving memory efficiency, using the *Three.js* framework to perform rendering rather than relying on heavy asset imports.

4.1 The Command Vessel

The central hub, representing the command vessel, is constructed as a hierarchical assembly of geometries. By utilizing *Three.js* group objects, complex structural forms are composed with local transformations. This modular approach allows for a complex model that looks like an actual massive spaceship without the overhead of complicated 3D meshes.

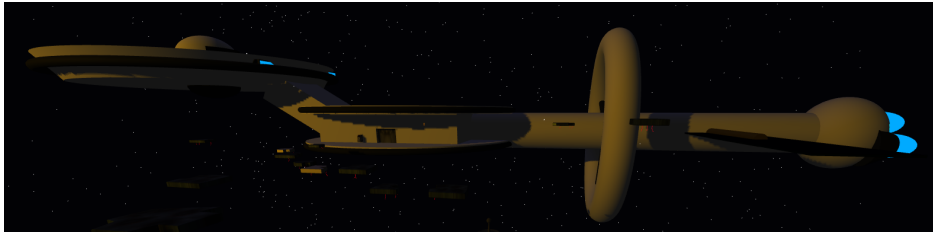


Figure 14: Vessel Model

4.2 The Black Hole

At the core of the background scene lies the supermassive black hole. Its accretion disk is rendered as a complex particle system that uses `THREE.BufferGeometry` to produce thousands of vertices distributed around the black hole. Each particle's position and color are calculated at runtime using trigonometric functions, which allows for a fluid animated appearance that simulates the relativistic effects of light spiraling toward the event horizon.

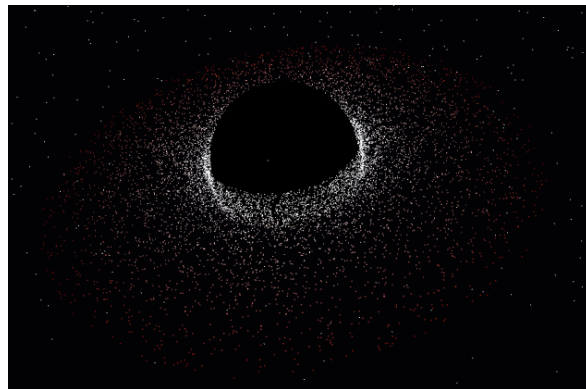


Figure 15: Black Hole

4.3 Galactic Clusters

Surrounding the spaceship are expansive galaxies, rendered as point-based systems. Every Galaxy contains 15000 points, randomly placed on a spiral shape, interpolate point sizes and color gradients, simulating the density of stellar formations. By treating galaxies as *BufferGeometry* clouds and not monolithic objects, the engine can efficiently cull off-screen particles and manage thousands of entities with minimal GPU pressure.

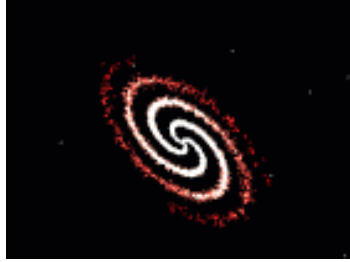


Figure 16: Galaxy

4.4 Sci-Fi Furnishings

The interior environment is populated with interactive objects and aesthetic modules. These decorations include:

- **Cryo-pods:** Modeled using geometric nesting to simulate glass reflections.
- **Mainframe:** Procedurally generated hardware with dynamic LED lights driven by periodic function updates.
- **Reactor Pedestal:** Altar that will support the reactor to end the game. Is produced by grouping cylinders and toruses .
- **Radar:** Scanning mechanism animated via random appearing points, that look like identified objects, constructed by grouping cylindrical primitive geometries.
- **Hologram:** Small 3D display of a solar system model rotating, composed with Three.js wiring with semi-transparent materials.
- **Wall Console:** Interactive control panel created by layering box primitives, serving as the physical interface for player interaction to read the logs for the incident.

4.5 Structural Foundation: Stars, Platforms, and Walls

The structural integrity of the environment is defined by its boundaries. The floor platforms and walls are composed of texture-mapped `THREE.BoxGeometry` instances.

To ensure visual continuity, a normalization algorithm manages texture offsets and scaling factors, preventing stretching artifacts regardless of the surface dimensions. Finally, the starfield is implemented as a sparse `THREE.Points` system that adds depth.

4.6 Textures

The textures applied on the models differ depending on the object. Most of the textures for the largest environment Models created in Three.js are monochromatic patterns. Smaller items, like the main room decorations, have some specific textures found online, in particular walls and pavements, to achieve variety on surfaces.

5 Animations

The project relies on a centralized `animate()` loop, synchronized with the display's refresh rate via `requestAnimationFrame()`, to handle all continuous transformations, physics calculations, and procedural animations within the scene. The entirety of the motion in this environment is calculated analytically in real-time, utilizing mathematical functions to simulate complex physical behaviors. No animation is imported.

5.1 Player Movements

The animation of the player character is driven entirely by code. The character's motion is simulated by manipulating the hierarchical nodes of its limbs (`leftArm`, `rightArm`, `leftLeg`, `rightLeg`).

When movement input is detected (via the WASD keys), the application applies a harmonic oscillation to these nodes. By utilizing a sinusoidal function dependent on the elapsed time (`Date.now() * speed`), the limbs rotate back and forth along the X-axis. The phases are inverted for opposite limbs (e.g., the right arm and left leg move backward while the left arm and right leg move forward), creating a natural walking or running cycle. The frequency of this oscillation is tied to the character's movement speed: when the player holds [Shift] to sprint, the limbs swing at a higher rate, so the animation visibly accelerates into a run. When the player ceases movement, a linear interpolation function smoothly returns the limbs to their default vertical positions. Furthermore, the player's vertical movement is governed by a basic physics simulation: a constant gravity value continuously decreases the `velocityY`, which is then applied to the player's Y-position, simulating falling and jumping arcs until a collision is detected.

5.2 The Solar System Simulation

The centerpiece of the environment's background is a dynamic animated solar system. The celestial bodies do not rely on static keyframes; instead, their orbital and rotational mechanics are driven by continuously updating rotation applied to hierarchical pivot groups.

The central star (`sunMesh`) rotates slowly on its Y-axis, accompanied by a separate particle system (`solarFlares`) representing solar activity, which rotates on multiple axes to simulate swirling plasma.

Four standard planets orbit the star at varying, constant angular velocities. This is achieved by parenting each planet to a centralized invisible Pivot object located at the star's coordinates, and rotating these pivots around the Y-axis. A moon is further animated by parenting it to a secondary pivot located at the center of the second planet, creating a compound orbital motion.

An artificial satellite is animated around one of the standard planets using the

same parenting technique, but with a tilted orbital plane (obtained by rotating its pivot on the X and Z axes before the continuous Y-axis rotation is applied). A point light attached to the satellite’s antenna blinks on and off.



Figure 17: Solar system planets’ animation

5.2.1 The Binary Planet System

A more complex celestial mechanic is implemented for a pair of binary planets (volcanic and glacial planets). While the pair orbits the central star via a common pivot, they also revolve around a shared barycenter. To simulate an approximation of Kepler’s Second Law, where bodies move faster when closer together and slower when further apart, the animation utilizes a trigonometric technique. A sine wave function ($\text{Math.sin}(1.5 * \text{currenttime})$) dictates both the rotational speed of the barycenter and the physical distance between the two planets. When the sine value approaches $+1$, the planets are drawn closer together and their angular velocity increases; conversely, as it approaches -1 , the distance expands and the rotation slows, creating a dynamic, “elastic” orbital effect.

5.2.2 Elliptical Comet Orbit and Tail Animation

The environment includes a comet that follows an elongated elliptical orbit, moving at a variable speed depending on its distance from the star. The orbit lies on the horizontal plane (XZ) and is defined by a semi-major axis $a = 550$ and a semi-minor axis $b = 320$.

To place the star (sunMesh) at the focus of the ellipse, rather than at its geometric center, the focal distance c is computed as:

$$c = \sqrt{a^2 - b^2} \quad (1)$$

This value is then subtracted from the x component of the orbit, so that the star sits exactly at one of the two foci instead of in the middle of the ellipse. The comet’s position $\mathbf{P}_c(\theta) = [x_c, y_c, z_c]^T$ is computed at each frame as a function of the orbital

angle θ :

$$x_c(\theta) = a \cos(\theta) - c \quad (2a)$$

$$y_c(\theta) = 0 \quad (2b)$$

$$z_c(\theta) = b \sin(\theta) \quad (2c)$$

The comet does not move at a constant angular speed: it speeds up near the star and slows down far from it, reproducing Kepler's Second Law in a simplified way. The current distance from the star is simply the length of the comet's position vector:

$$r(t) = \|\mathbf{P}_c(t)\| = \sqrt{x_c^2 + z_c^2} \quad (3)$$

and the orbital speed $\omega(t)$ is set inversely proportional to this distance:

$$\omega(t) = \frac{\alpha}{r(t)}, \quad \alpha = 20 \quad (4)$$

where α is a constant tuned to obtain a visually convincing speed variation. At every frame, the orbital angle θ is incremented by $\omega(t)$, scaled by a small fixed factor (0.02) to keep the motion smooth and frame-rate independent:

$$\theta_{t+\Delta t} = \theta_t + \omega(t) \cdot 0.02 \quad (5)$$

The comet's tail is kept oriented away from the star at every frame using Three.js's `lookAt()` function, so that the tail consistently points away from the star regardless of the comet's position. Its length also reacts to the distance from the star, stretching when the comet gets closer and shrinking when it moves away, simulating the effect of solar wind:

$$S_z(t) = 0.4 + \frac{\beta}{r(t)}, \quad \beta = 200 \quad (6)$$

Finally, a few lightweight effects are layered on top to make the tail feel more like flowing plasma rather than a static shape. The tail continuously rotates around its own axis at a constant rate, giving the impression of swirling particles. and its cross-section pulses slightly over time using two independent sine waves, one for each axis, so the tail appears to flutter rather than expand and contract uniformly:

$$S_x(t) = 1.0 + 0.1 \cdot \sin(0.004 \cdot t) \quad (7a)$$

$$S_y(t) = 1.0 + 0.1 \cdot \cos(1.2 \cdot 0.004 \cdot t) \quad (7b)$$

Using a slightly different frequency for S_y (the 1.2 multiplier) avoids the two axes pulsing in perfect sync, which would look mechanical.



Figure 18: Comet animation

5.3 Black Hole Disk Dynamics

The black hole is surrounded by an accretion disk made of $N = 10,000$ individual particles. Instead of treating each particle as a separate Three.js object, the disk is implemented as a single BufferGeometry, and the particle positions are updated directly by editing the position array on every frame. This avoids the overhead of managing 10,000 individual objects and keeps the simulation fast enough to run in real time.

For each particle $i \in [0, N - 1]$, the position in the orbital plane $\mathbf{x}_i = [x_i, z_i]^T$ is read from the flat position array, at indices i and $i + 2$ (the Y component, at index $i + 1$, is left untouched since the disk is flat). The distance of the particle from the center of the disk is computed with the standard 2D distance formula:

$$r_i = \sqrt{x_i^2 + z_i^2} \quad (8)$$

To make particles closer to the center rotate visibly faster than particles further out, mimicking the way matter behaves near a real black hole, each particle's angular speed $\Omega(r_i)$ is set inversely proportional to its distance from the center, raised to the power of 1.5:

$$\Omega(r_i) = \frac{\mu}{r_i^{1.5}}, \quad \mu = 15 \quad (9)$$

where μ is an empirical constant used to tune the overall speed of the rotation. Because of this relationship, particles near the center complete a full orbit much faster than particles on the outer edge, giving the disk its characteristic swirling appearance.

Once $\Omega(r_i)$ is known, the particle's position is rotated by that angle using a standard 2D rotation matrix (the negative sign fixes the rotation direction to clockwise):

$$\begin{bmatrix} x_i \\ z_i \end{bmatrix}_{t+\Delta t} = \begin{bmatrix} \cos(-\Omega(r_i)) & -\sin(-\Omega(r_i)) \\ \sin(-\Omega(r_i)) & \cos(-\Omega(r_i)) \end{bmatrix} \begin{bmatrix} x_i \\ z_i \end{bmatrix}_t \quad (10)$$

which expands to the following per-particle update, applied directly to the position array:

$$x_i(t + \Delta t) = x_i(t) \cos(\Omega(r_i)) - z_i(t) \sin(\Omega(r_i)) \quad (11a)$$

$$z_i(t + \Delta t) = x_i(t) \sin(\Omega(r_i)) + z_i(t) \cos(\Omega(r_i)) \quad (11b)$$

After all 10,000 particles have been updated, the `needsUpdate` flag of the position attribute is set to `true`. This tells Three.js that the buffer has changed and that the new positions need to be re-uploaded to the GPU before the next frame is rendered.

5.4 Gravity and collisions

The gravitational pull in the environment is simulated through a continuous update applied at each frame in the main rendering loop. A constant downward acceleration, defined by the gravity coefficient, is accumulated into the player's vertical velocity vector (`velocityY`). This velocity is subsequently added to the player's Y -coordinate, simulating a realistic free-fall.

To prevent the character from falling indefinitely, this vertical translation is coupled with the scene's collision detection system. The algorithm iterates through all walkable surfaces, such as dynamic platforms and the central hull disk, performing box intersection checks. If a collision is detected beneath the player's feet, the `onObject` flag is triggered: the vertical velocity is immediately zeroed, the jumping state is reset, and the player's Y -coordinate is locked to the top level of the intersected geometry, effectively simulating the normal force of the ground.

Furthermore, a fail-safe condition is implemented: should the player fall below a depth threshold ($Y < -30$), the system resets the spatial coordinates to a safe spawn point, preventing the character from falling into the void. This reset is accompanied by a brief message, framed as an emergency teleport rather than a failure screen. The message is a HTML overlay that fades in and out automatically over a couple of seconds through a CSS animation.

Collisions with the vertical walls are implemented in a similar way to the horizontal platforms.

5.5 Camera Management and Anti-Clipping System

To guarantee a third-person perspective and prevent the camera from clipping through environmental geometry, a dynamic clipping-avoidance system based on raycasting has been implemented. Rather than attaching the camera to the player's local coordinates, the camera operates independently and follows a calculated target.

At each frame, the system computes an *ideal position* for the camera based on an optimal distance and height relative to the player, adjusted by the current angles dictated by the user's mouse input. A `THREE.Raycaster` is then cast from the player's center of mass outwards towards this ideal spatial coordinate. The raycaster evaluates intersections exclusively against the solid objects. If an occluding obstacle is detected between the player and the ideal camera position, the algorithm recalculates a safe offset, setting the camera's final position slightly in front of the intersected face to maintain a clear line of sight. Finally, to prevent visual snaps the camera's actual

spatial coordinates are smoothly transitioned towards the evaluated target position using a Linear Interpolation mathematical function. Its directional vector is constantly forced to track the player using the lookAt method.

5.6 Environmental and UI Animations

The environment contains numerous small animations that enhance the scene's dynamism:

- **Tweening:** Discrete state changes, such as the pressing of interactive buttons or the opening of doors, are managed by the TWEEN.js library. This allows for smooth transitions over time without manually managing increments.
- **Floating and Oscillating Elements:** items like crystals and specific platforms feature a continuous oscillation achieved by adding the result of a `Math.sin()` function to their Y-position.
- **Material animations:** The opacity of the star field background oscillates using a sine wave to create a twinkling effect. Similarly, interactive UI elements, such as the radar blip, modulate their opacity over time, with the blip relocating to a new random coordinate periodically, creating the illusion of scanning.
- **Galactic Rotation:** Distant background galaxies are given a very slow, constant rotation to provide a sense of continuous motion in the deep background.
- **UI Effects:** UI HTML elements are used to add variety. Many different HTML elements pop in and out and have oscillating intensity texts. Also, the final victory animation combines many kinds of animations: the player moving his arms up and down cheering, the camera rotating around it and a UI element appearing saying "Mission Accomplished"
- **Props Animations:** Many small models have animations hidden in them. For example, lights in the mainframe turn on and off randomly, the screen of the log console shows a moving screen, the central hologram shows a small rotating solar system and the pipes on the walls have moving textures, to simulate flowing plasma inside them.

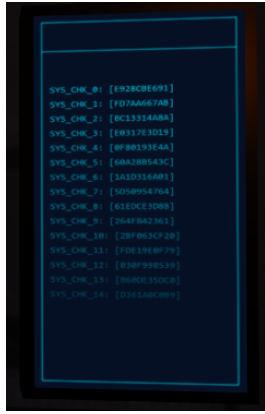


Figure 19: Log console screen animation



Figure 20: Mainframe lights animation

6 Lighting

The lighting system is designed to achieve a balance between a dark atmospheric sci-fi aesthetic, lighting mechanics and good performance. The scene does not rely on a single global light, but rather combines different types of illumination to define the space and enable gameplay mechanics. The lighting architecture can be categorized into three main layers: Global Illumination, Decorative Lighting, and Dynamic Interactive Lights.

6.1 Global Illumination and Atmospheric Lighting

To establish the baseline visibility without removing the shadows necessary for the space-station atmosphere, two global light sources are instantiated:

- **Ambient Light** (`THREE.AmbientLight`): A very low-intensity white light (0.1) is applied globally. This light does not have a specific direction and does not cast shadows. Its sole purpose is to ensure that areas completely hidden from direct light sources do not appear entirely pitch black, allowing the player to understand and see the basic structure of the environment.
- **Star Light** (`THREE.PointLight`): Simulating the light from a distant star, a yellow pointlight is cast from the distant sun of the solar system. It mainly provides light for the planets that would be otherwise invisible in the black space, but also some visibility for the outside platforming.

6.2 Local and Decorative Lighting

Throughout the spaceship, specific `THREE.PointLight` objects are utilized to emphasize key elements.

- **Main room Ambient Light**: A central `PointLight` is placed inside the ceiling of the main room. Unlike global lights, it illuminates uniformly in all directions but its intensity decays with distance, acting as the primary source of illumination for the main chamber but not interfering with the light activated puzzles. To maintain a smooth framerate, shadow calculations are disabled for this light, allowing it to act purely as a diffuse light to allow the player to see the room.
- **Aesthetic Micro-Lights**: Minor `PointLight` sources with extremely short decay distances and low intensities are positioned inside specific props, such as the engine and floor vents. These lights do not contribute to the illumination of the room but serve purely as emissive accents to enhance the machinery.

6.3 Dynamic Player-Controlled Lighting (The Flashlight)

The primary tool for both exploration and puzzle-solving is the player’s portable lamp. This is implemented using a `THREE.SpotLight` attached to the player. The spotlight projects a focused cone of light that follows the camera’s orientation. The player can toggle this light on and off, directly interacting with the environment. Because this is the main focus of the gameplay, it is the main light source that actively triggers game logic based on what it illuminates. There’s another light source that can interact with light activated puzzles, and that is an interactive `THREE.PointLight` located over one of the light sensors. It can be activated by the player pressing of the buttons.

6.4 Physical Lighting Logic and Intensity Computation

To make game elements react dynamically and realistically to light sources, the lighting logic is built around a simplified ray-tracing approach: rather than relying purely on the renderer’s shading, the actual intensity reaching a specific object is computed manually, combining distance, angle, and visibility checks.

The algorithm takes a light source L and a target object T as input, and determines the intensity reaching T through three steps:

1. **Inverse Square Law Attenuation:** The distance d between the light’s world position $\mathbf{p}_L$ and the target’s world position $\mathbf{p}_T$ is computed. As a first optimization, if d exceeds a fixed threshold of 20 units, the function returns zero immediately, without performing any further calculation, because it’s too far. Otherwise, the base intensity is attenuated following the standard inverse-square falloff used for real light sources:

$$I_{\text{base}} = \frac{I_0}{d^2} \quad (12)$$

where I_0 is the source’s nominal intensity.

2. **Spotlight Cone Modeling:** If the light source is a spotlight (*Three.SpotLight*), the algorithm checks whether the target actually falls inside the light’s cone. This is done by computing the dot product $(\cdot)$ between the lamp’s direction vector $\mathbf{d}_{\text{lamp}}$ (aligned with the player’s orientation) and the normalized direction vector $\mathbf{v}_{\text{dir}}$ pointing from the light towards the target. Since the dot product of two normalized vectors equals the cosine of the angle between them, the target lies inside the cone whenever:

$$\mathbf{d}_{\text{lamp}} \cdot \mathbf{v}_{\text{dir}} \geq \cos(\alpha) \quad (13)$$

where α is the spotlight’s cutoff angle; if this condition fails, the object is outside the beam and the intensity is set to zero. To avoid a hard edge at the border of the cone, the dot product is remapped into a $[0, 1]$ range (where 0 is the exact

edge of the cone and 1 is looking straight at the target) and raised to a power controlled by the spotlight’s *penumbra* parameter. This produces a smooth falloff near the edges of the beam instead of a cutoff: a low *penumbra* value gives a gradual fade, while a high value gives a sharper transition.

3. **Visibility Verification via Shadow Rays:** Finally, to check whether something is physically blocking the light before it reaches the target, a secondary ray (a *shadow ray*) is cast using `THREE.Raycaster`. The ray originates at $\mathbf{p}_L$, travels along the direction $\mathbf{v}_{\text{dir}}$, and has a length of $d + 0.5$.

A key aspect of this implementation is the optimization used to keep this check cheap enough to run every frame for many light–object pairs. Instead of testing the shadow ray against the entire scene, which would include thousands of irrelevant decorative meshes, the raycaster is restricted to a small set $\mathcal{O}$ containing only the geometries that can actually cast a shadow:

$$\mathcal{O} = \{\text{walls}\} \cup \{\text{platforms}\} \cup \{T\} \quad (14)$$

If the first object hit by the ray is neither the target T nor its parent, it means something else (typically a wall or a platform) is standing between the light and the target. In that case the function returns an intensity of zero, effectively casting a shadow on the object.

7 Audio

To reinforce the atmosphere and give feedback to the player's actions, the application features an audio system built entirely on the native Three.js audio classes. A single `THREE.AudioListener` is attached to the camera and acts as the virtual "ears" of the scene. Every sound is routed through this listener, whose volume can be modified by the player in the pause menu.

Because browsers suspend the audio until the user interacts with the page, playback is unlocked on the first click on the game canvas, which also resumes the context and starts the looping background track. The audio is organized into three categories.

7.1 Ambient Music

A continuous background track is played as a non-positional audio source, looping for the entire duration of the session at a low volume to establish the atmosphere without covering the sound effects. During pause, its volume is lowered rather than cut entirely, since the game is suspended.

7.2 Positional Audio

The mainframe emits a continuous electrical buzz implemented as a `PositionalAudio` source. Unlike the ambient music, this sound is attached directly to the mainframe in the scene, so its perceived volume is computed based on the distance between the listener and the object: the buzz grows louder as the player approaches the machine and fades as they move away, giving depth to the room.

7.3 Sound Effects

The remaining sounds are short, non-positional effects triggered by gameplay events. Each effect is loaded once by the `init()` function and played on demand, restarting the sample if it is triggered again in quick succession. The sound effects cover the main interactions of the game: the pressing of buttons, the opening and closing of the door, the flashlight toggle, the pickup and placement of the reactor, the respawn after a fall, the footsteps during movement, and the final victory jingle. Footsteps in particular are not played once per frame but gated behind a timer whose interval shortens when the player is sprinting, so that the cadence of the steps matches the character's movement speed.